Roteiro de estudo

MySQL: subconsultas

Márcio Fantini - coltec@ufmg.br

Tipos de retorno, operadores, correlação e tabelas derivadas

Continuação dos roteiros de MySQL básico e junções

Sumário

1 Apresentação e objetivos **1**

2 O que é uma subconsulta? **2**
2.1 Subconsulta independente e correlacionada 2

3 Banco de dados utilizado no roteiro **3**
3.1 Os três valores relacionados ao produto 3
3.2 Criando banco e tabelas 3
3.3 Inserindo os clientes 4
3.4 Inserindo os produtos 5
3.5 Inserindo as vendas 5

4 A questão central: qual é o formato do resultado? **6**
4.1 Os quatro formatos fundamentais 6
4.2 Um valor: uma linha e uma coluna 7
4.3 Uma coluna: várias linhas 7
4.4 Uma linha: várias colunas 7
4.5 Uma tabela: várias linhas e várias colunas 8

5 Subconsultas escalares: um único valor **8**
5.1 Produtos acima do preço médio 8
5.2 Produto mais caro 9
5.3 Vendas maiores do que a média das vendas 9
5.4 Preço de um produto específico 9
5.5 E se a subconsulta não retornar nenhuma linha? 10
5.6 E se ela retornar várias linhas? 10

6 Subconsultas de coluna: várias linhas e uma coluna **11**
6.1 Clientes que já realizaram compras 11
6.2 Produtos que já foram vendidos 12
6.3 Clientes que pagaram com Pix 12
6.4 Produtos comprados por clientes mineiros 12
6.5 DISTINCT é obrigatório? 13

7 NOT IN e o problema dos valores nulos **13**
7.1 Clientes sem compras 13
7.2 Produtos nunca vendidos 14

7.3 Por que um NULL pode invalidar o resultado? . . . 14
7.4 Primeira solução: eliminar os nulos . . . 14
7.5 Segunda solução: preferir NOT EXISTS . . . 15

8 Comparações com ANY, SOME e ALL 15
8.1 Maior que algum produto de Acessórios . . . 16
8.2 Maior que todos os produtos de Acessórios . . . 16
8.3 Relação com MIN e MAX . . . 16

9 Subconsultas de linha: várias colunas em uma linha 17
9.1 Mesma categoria e mesmo preço de um produto . . . 17
9.2 Mesmo cliente e produto de uma venda . . . 18
9.3 O que acontece quando as quantidades não coincidem? . . . 18

10 Subconsultas com várias linhas e várias colunas 19
10.1 Tuplas com IN . . . 19
10.2 Venda mais recente de cada cliente . . . 20

11 Subconsultas no FROM: tabelas derivadas 21
11.1 Total gasto por cliente . . . 21
11.2 Filtrando um relatório intermediário . . . 21
11.3 Calculando a média dos totais por cliente . . . 22
11.4 Juntando a tabela derivada com clientes . . . 22
11.5 Incluindo clientes sem compras . . . 22

12 Subconsultas correlacionadas 23
12.1 Produtos acima da média de sua própria categoria . . . 24
12.2 Produto mais caro de cada categoria . . . 24
12.3 Venda mais recente de cada cliente . . . 25
12.4 Venda acima da média do próprio produto . . . 25

13 EXISTS e NOT EXISTS 26
13.1 Clientes que já compraram . . . 26
13.2 Clientes que nunca compraram . . . 26
13.3 Produtos nunca vendidos . . . 26
13.4 Clientes que compraram produtos de Telefonia . . . 27
13.5 Por que escrever SELECT 1? . . . 27

14 Subconsultas na lista do SELECT 28
14.1 Quantidade de compras por cliente . . . 28

14.2 Total gasto por cliente . . . 28
14.3 Data da última compra . . . 29
14.4 Diferença entre COUNT e SUM em conjunto vazio . . . 29

15 Subconsultas no HAVING 30
15.1 Categorias acima da média dos preços gerais . . . 30
15.2 Clientes com gasto acima da média dos totais . . . 30
15.3 Categorias com faturamento acima do faturamento médio . . . 30

16 Como escolher a subconsulta correta 31
16.1 Fluxo de decisão . . . 32
16.2 O mesmo problema visto por abordagens diferentes . . . 33

17 Erros frequentes 33
17.1 Erro 1: várias linhas onde se espera uma . . . 33
17.2 Erro 2: várias colunas onde se espera uma . . . 34
17.3 Erro 3: esquecer o alias de uma tabela derivada . . . 34
17.4 Erro 4: usar NOT IN com valores nulos . . . 35
17.5 Erro 5: usar LIMIT 1 sem critério . . . 35
17.6 Erro 6: esquecer a correlação . . . 35
17.7 Erro 7: confundir média de vendas com média de clientes . . . 36

18 Subconsultas e junções: comparação 36
18.1 Clientes que compraram . . . 37
18.2 Clientes que nunca compraram . . . 37
18.3 Produto mais caro por categoria . . . 37

19 Revisão geral: estudo dirigido 38

20 Gabarito sugerido 39
20.1 Atividade 4.1 . . . 40
20.2 Atividade 5.1 . . . 40
20.3 Atividade 6.1 . . . 40
20.4 Atividade 7.1 . . . 41
20.5 Atividade 8.1 . . . 42
20.6 Atividade 9.1 . . . 43
20.7 Atividade 10.1 . . . 43
20.8 Atividade 11.1 . . . 44
20.9 Atividade 12.1 . . . 45

20.10 Atividade 13.1 46
20.11 Atividade 14.1 47
20.12 Atividade 15.1 47
20.13 Revisão 19.1 — classificação 48
20.14 Revisão 19.2 — subconsultas escalares 48
20.15 Revisão 19.3 — listas, presença e ausência 49
20.16 Revisão 19.4 — correlação e agrupamentos 50
20.17 Revisão 19.5 — desafios integradores 51

21 Checklist do estudante **52**

1 Apresentação e objetivos

Uma subconsulta permite que uma consulta utilize o resultado de outra. Esse recurso ajuda a formular perguntas que exigem uma informação intermediária:

- Quais produtos custam mais do que a média de todos os produtos?
- Quais clientes já realizaram pelo menos uma compra?
- Quais produtos nunca foram vendidos?
- Qual é o produto mais caro de cada categoria?
- Quais clientes gastaram acima da média dos totais por cliente?

Objetivos de aprendizagem

Ao final do roteiro, o estudante deverá ser capaz de:

- identificar a consulta principal e a subconsulta;
- classificar resultados como escalares, colunas, linhas ou tabelas;
- escolher operadores compatíveis com o formato retornado;
- utilizar `=`, `>`, `<`, `IN` e `NOT IN`;
- compreender `ANY`, `ALL`, `EXISTS` e `NOT EXISTS`;
- construir subconsultas no `SELECT`, no `WHERE`, no `HAVING` e no `FROM`;
- diferenciar subconsultas independentes e correlacionadas;
- interpretar erros de quantidade de linhas e colunas;
- reconhecer problemas causados por `NULL`;
- comparar soluções com subconsultas e soluções com junções.

Aula	Conteúdo principal	Referência
1	Conceitos, modelo do banco e formatos de retorno	Seções 2 a 4
2	Subconsultas escalares e de uma coluna	Seções 5 a 7
3	Comparações com `ANY`, `ALL` e subconsultas de linha	Seções 8 e 9
4	Subconsultas tabulares e tabelas derivadas	Seções 10 e 11
5	Subconsultas correlacionadas, `EXISTS` e `NOT EXISTS`	Seções 12 e 13
6	Subconsultas no `SELECT` e no `HAVING`	Seções 14 e 15
7	Escolha da técnica, erros comuns e comparação com junções	Seções 16 a 18
8	Estudo dirigido e gabarito	Seções 19 e 20

2 O que é uma subconsulta?

Uma **subconsulta**, também chamada *subquery*, é uma instrução `SELECT` colocada dentro de outra instrução SQL.

Código 1: Estrutura geral de uma subconsulta

```sql
SELECT colunas
FROM tabela_principal
WHERE coluna OPERADOR (
    SELECT coluna_ou_expressao
    FROM outra_tabela
    WHERE condicao
);
```

Duas partes da mesma pergunta

A consulta externa formula a pergunta principal. A consulta interna produz uma informação necessária para responder a essa pergunta.

Considere a pergunta:

> Quais produtos custam mais do que o preço médio dos produtos?

Podemos separá-la em dois passos conceituais:

1. descobrir o preço médio;
2. selecionar produtos cujo preço seja maior que essa média.

Código 2: Primeiro exemplo completo

```sql
SELECT
    nome_produto,
    preco
FROM produtos
WHERE preco > (
    SELECT AVG(preco)
    FROM produtos
);
```

Leia o resultado

A expressão interna calcula um único número. A expressão externa compara o preço de cada produto com esse número.

2.1 Subconsulta independente e correlacionada

Uma subconsulta **independente** não utiliza colunas da consulta externa. Em termos didáticos, ela pode ser analisada separadamente.

Código 3: Subconsulta independente

```sql
SELECT AVG(preco)
FROM produtos;
```

Uma subconsulta **correlacionada** depende de uma coluna da consulta externa:

Código 4: Subconsulta correlacionada

```sql
SELECT
    p1.nome_produto,
    p1.categoria,
    p1.preco
FROM produtos AS p1
WHERE p1.preco > (
    SELECT AVG(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);
```

Modelo mental de execução

Conceitualmente, a subconsulta correlacionada é avaliada em relação a cada linha externa. O otimizador do MySQL pode reorganizar internamente a execução, mas o significado lógico permanece o mesmo.

3 Banco de dados utilizado no roteiro

Reutilizaremos o modelo do roteiro de junções: `clientes`, `produtos` e `vendas`. Como discutido anteriormente, a tabela de produtos também terá `custo_aquisicao`.

3.1 Os três valores relacionados ao produto

Campo	Significado
`produtos.custo_aquisicao`	valor pago pela loja ao fornecedor;
`produtos.preco`	preço atual de venda anunciado no catálogo;
`vendas.valor_unitario`	preço efetivamente cobrado naquela venda.

Preservando informações diferentes

O custo mostra quanto o produto custou à loja. O preço mostra quanto a loja pretende cobrar atualmente. O valor unitário da venda preserva o preço efetivamente praticado em uma compra específica.

3.2 Criando banco e tabelas

Código 5: Banco de estudo e tabela clientes

```sql
CREATE DATABASE loja_subconsultas
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE loja_subconsultas;

CREATE TABLE clientes (
    id_cliente     INT PRIMARY KEY AUTO_INCREMENT,
    nome           VARCHAR(100) NOT NULL,
    cidade         VARCHAR(60) NOT NULL,
    estado         CHAR(2) NOT NULL,
    data_cadastro  DATE NOT NULL,
    ativo          BOOLEAN NOT NULL DEFAULT TRUE
);
```

Código 6: Tabela produtos com custo e preço atual

```sql
CREATE TABLE produtos (
    id_produto       INT PRIMARY KEY AUTO_INCREMENT,
    nome_produto     VARCHAR(100) NOT NULL,
    categoria        VARCHAR(50) NOT NULL,
    custo_aquisicao  DECIMAL(10,2) NOT NULL,
    preco            DECIMAL(10,2) NOT NULL,
    estoque          INT NOT NULL,
    ativo            BOOLEAN NOT NULL DEFAULT TRUE
);
```

Código 7: Tabela vendas com valor histórico

```sql
CREATE TABLE vendas (
    id_venda         INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente       INT NOT NULL,
    id_produto       INT NOT NULL,
    quantidade       INT NOT NULL,
    valor_unitario   DECIMAL(10,2) NOT NULL,
    data_venda       DATE NOT NULL,
    forma_pagamento  VARCHAR(30) NOT NULL,

    CONSTRAINT fk_sub_vendas_clientes
      FOREIGN KEY (id_cliente)
      REFERENCES clientes (id_cliente),

    CONSTRAINT fk_sub_vendas_produtos
      FOREIGN KEY (id_produto)
      REFERENCES produtos (id_produto)
);
```

3.3 Inserindo os clientes

Código 8: Doze clientes cadastrados

```sql
INSERT INTO clientes
  (nome, cidade, estado, data_cadastro, ativo)
VALUES
('Ana Souza',       'Belo Horizonte', 'MG', '2025-01-03', TRUE),
('Bruno Lima',      'Contagem',       'MG', '2025-01-08', TRUE),
('Carla Mendes',    'São Paulo',      'SP', '2025-01-12', TRUE),
('Daniel Rocha',    'Rio de Janeiro', 'RJ', '2025-01-20', TRUE),
('Elisa Martins',   'Belo Horizonte', 'MG', '2025-02-01', TRUE),
('Fábio Costa',     'Salvador',       'BA', '2025-02-09', TRUE),
('Gabriela Alves',  'Curitiba',       'PR', '2025-02-15', FALSE),
('Henrique Nunes',  'São Paulo',      'SP', '2025-03-02', TRUE),
('Isabela Freitas', 'Vitória',        'ES', '2025-03-10', TRUE),
('João Ribeiro',    'Belo Horizonte', 'MG', '2025-03-22', TRUE),
('Karen Oliveira',  'Recife',         'PE', '2025-04-05', TRUE),
('Lucas Pereira',   'Contagem',       'MG', '2025-04-18', TRUE);
```

3.4 Inserindo os produtos

Código 9: Doze produtos com custo e preço

```sql
INSERT INTO produtos
  (nome_produto, categoria, custo_aquisicao,
   preco, estoque, ativo)
VALUES
('Notebook Pro 14',     'Informática', 3300.00, 4590.00,  8, TRUE),
('Mouse sem fio',       'Acessórios',    65.00,  120.00, 35, TRUE),
('Monitor 24',          'Informática',  690.00,  980.00, 12, TRUE),
('Teclado mecânico',    'Acessórios',   210.00,  350.00, 20, TRUE),
('Smartphone X',        'Telefonia',   2050.00, 2850.00, 10, TRUE),
('Webcam HD',           'Acessórios',   125.00,  210.00, 18, TRUE),
('Tablet 10',           'Informática', 1190.00, 1750.00,  9, TRUE),
('Fone Bluetooth',      'Telefonia',    105.00,  190.00, 25, TRUE),
('SSD 1 TB',            'Informática',  340.00,  520.00, 14, TRUE),
('Cadeira ergonômica',  'Escritório',   930.00, 1350.00,  6, TRUE),
('Impressora térmica',  'Escritório',   510.00,  780.00,  5, TRUE),
('Projetor multimídia', 'Informática', 1700.00, 2390.00,  4, TRUE);
```

3.5 Inserindo as vendas

Código 10: Dezoito vendas de exemplo

```sql
INSERT INTO vendas
  (id_cliente, id_produto, quantidade, valor_unitario,
   data_venda, forma_pagamento)
VALUES
(1,  1, 1, 4590.00, '2025-01-08', 'Cartão'),
(2,  2, 2,  120.00, '2025-01-10', 'Pix'),
(3,  3, 1,  980.00, '2025-01-15', 'Cartão'),
```

```
(5,  4, 1,  350.00, '2025-02-03', 'Pix'),
(1,  2, 1,  120.00, '2025-02-12', 'Dinheiro'),
(6,  5, 1, 2850.00, '2025-02-21', 'Cartão'),
(8,  6, 2,  210.00, '2025-03-02', 'Pix'),
(9,  7, 1, 1750.00, '2025-03-18', 'Boleto'),
(10, 8, 3,  190.00, '2025-03-25', 'Pix'),
(12, 3, 1,  980.00, '2025-04-07', 'Cartão'),
(2,  4, 1,  350.00, '2025-04-16', 'Dinheiro'),
(3,  5, 1, 2750.00, '2025-04-29', 'Cartão'),
(5,  9, 2,  520.00, '2025-05-06', 'Pix'),
(6,  8, 2,  190.00, '2025-05-20', 'Boleto'),
(8, 10, 1, 1350.00, '2025-06-04', 'Cartão'),
(9,  2, 3,  115.00, '2025-06-18', 'Pix'),
(10, 9, 1,  520.00, '2025-07-01', 'Boleto'),
(1,  6, 1,  210.00, '2025-07-12', 'Pix');
```

Atenção

Daniel, Gabriela e Karen não realizaram compras. Impressora térmica e Projetor multimídia ainda não foram vendidos. Esses registros serão importantes em exemplos com `NOT IN` e `NOT EXISTS`.

4 A questão central: qual é o formato do resultado?

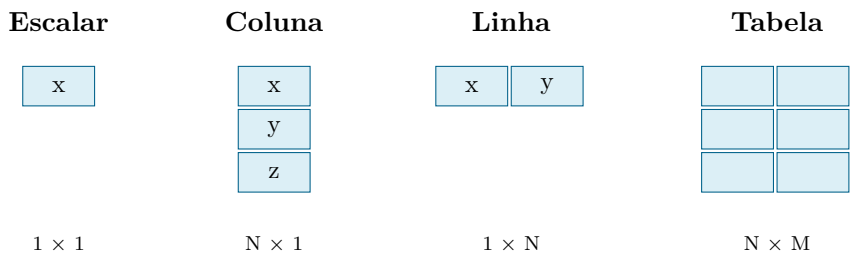
A dúvida mais comum não é somente o que a subconsulta retorna, mas também **quantas linhas e quantas colunas** podem ser retornadas.

Regra de ouro

O operador ou a posição em que a subconsulta é usada determina qual formato de resultado é permitido. Antes de escolher a sintaxe, pergunte: a consulta interna produz uma célula, uma coluna, uma linha ou uma tabela?

4.1 Os quatro formatos fundamentais

Formato	Dimensão	Uso mais comum
Escalar	1 linha e 1 coluna	comparações simples, expressões e colunas calculadas;
Coluna	várias linhas e 1 coluna	`IN`, `NOT IN`, `ANY`, `ALL`;
Linha	1 linha e várias colunas	comparação de tuplas, como `(a,b) = (...)` ;
Tabela	várias linhas e várias colunas	`FROM`, tabelas derivadas e comparação de tuplas com `IN`.



4.2 Um valor: uma linha e uma coluna

Código 11: Subconsulta escalar

```
SELECT MAX(preco)
FROM produtos;
```

Resultado conceitual:

MAX(preco)
4590,00

4.3 Uma coluna: várias linhas

Código 12: Subconsulta de uma coluna

```
SELECT id_cliente
FROM vendas
WHERE forma_pagamento = 'Pix';
```

Resultado conceitual: uma lista de identificadores. Um mesmo identificador pode aparecer mais de uma vez.

4.4 Uma linha: várias colunas

Código 13: Subconsulta de linha

```
SELECT categoria, preco
FROM produtos
WHERE id_produto = 3;
```

Resultado conceitual:

categoria	preco
Informática	980,00

4.5 Uma tabela: várias linhas e várias colunas

Código 14: Subconsulta tabular

```sql
SELECT
    id_cliente,
    SUM(quantidade * valor_unitario) AS total_gasto
FROM vendas
GROUP BY id_cliente;
```

Aqui cada linha representa um cliente, e as duas colunas representam seu identificador e seu total gasto.

Qual tipo devo usar?

- Quer comparar com um único número? Use uma subconsulta escalar.
- Quer verificar se um valor pertence a uma lista? Use `IN`.
- Quer comparar duas ou mais colunas ao mesmo tempo? Use tuplas.
- Quer consultar uma tabela intermediária? Use uma subconsulta no `FROM`.
- Quer saber se existe alguma linha relacionada? Use `EXISTS`.

Atividade 4.1

Classifique o formato retornado por cada consulta:

a) `SELECT COUNT(*) FROM clientes;`

b) `SELECT id_cliente FROM vendas;`

c) `SELECT nome, estado FROM clientes WHERE id_cliente = 1;`

d) `SELECT id_cliente, COUNT(*) FROM vendas GROUP BY id_cliente;`

e) `SELECT AVG(preco) FROM produtos WHERE categoria = 'Telefonia';`

5 Subconsultas escalares: um único valor

Uma subconsulta escalar retorna **uma coluna e no máximo uma linha**. Ela pode ser utilizada onde uma expressão comum é esperada.

Quando escolher uma subconsulta escalar

Use esse tipo quando a pergunta depender de um único valor: uma média, um maior preço, uma menor data, uma contagem ou um identificador único.

5.1 Produtos acima do preço médio

Código 15: Comparação com um único número

```sql
SELECT
    nome_produto,
    preco
FROM produtos
WHERE preco > (
    SELECT AVG(preco)
    FROM produtos
)
ORDER BY preco DESC;
```

5.2 Produto mais caro

Código 16: Comparação com o preço máximo

```sql
SELECT
    nome_produto,
    categoria,
    preco
FROM produtos
WHERE preco = (
    SELECT MAX(preco)
    FROM produtos
);
```

Leia o resultado

Se vários produtos tiverem exatamente o mesmo maior preço, todos aparecerão. A subconsulta continua escalar porque `MAX(preco)` retorna um valor.

5.3 Vendas maiores do que a média das vendas

Código 17: Comparando o total de cada item com a média

```sql
SELECT
    id_venda,
    quantidade,
    valor_unitario,
    quantidade * valor_unitario AS total
FROM vendas
WHERE quantidade * valor_unitario > (
    SELECT AVG(quantidade * valor_unitario)
    FROM vendas
)
ORDER BY total DESC;
```

5.4 Preço de um produto específico

Código 18: Produtos mais caros que o Monitor 24

```sql
SELECT
    nome_produto,
    preco
FROM produtos
WHERE preco > (
    SELECT preco
    FROM produtos
    WHERE id_produto = 3
)
ORDER BY preco;
```

Por que a consulta interna retorna apenas uma linha?

O filtro utiliza `id_produto`, uma chave primária. Como a chave é única, no máximo uma linha poderá satisfazer a condição.

5.5 E se a subconsulta não retornar nenhuma linha?

Código 19: Subconsulta escalar sem correspondência

```sql
SELECT
    nome_produto,
    preco
FROM produtos
WHERE preco > (
    SELECT preco
    FROM produtos
    WHERE id_produto = 999
);
```

Como não existe produto 999, a subconsulta escalar produz `NULL`. A comparação `preco > NULL` não é verdadeira; portanto, a consulta não retorna produtos.

5.6 E se ela retornar várias linhas?

Código 20: Exemplo incorreto: operador espera um valor

```sql
SELECT nome_produto, preco
FROM produtos
WHERE preco = (
    SELECT preco
    FROM produtos
    WHERE categoria = 'Informática'
);
```

Há vários produtos de Informática. Como `=` exige um único valor nesse contexto, o MySQL informa:

```
ERROR 1242 (21000): Subquery returns more than 1 row
```

Atenção

Para corrigir, primeiro defina a pergunta correta. Se a intenção é pertencer a uma lista, use `IN`. Se a intenção é comparar com um único resumo, use `MAX`, `MIN` ou `AVG`. Não adicione `LIMIT 1` sem definir um critério que faça sentido.

Atividade 5.1

Escreva subconsultas escalares para:

a) listar produtos acima da média dos preços;

b) encontrar o produto de menor preço;

c) encontrar vendas realizadas na data mais recente;

d) listar produtos com custo acima do custo médio;

e) encontrar produtos cuja margem `preco - custo_aquisicao` esteja acima da margem média.

6 Subconsultas de coluna: várias linhas e uma coluna

Uma subconsulta de coluna retorna exatamente **uma coluna**, mas pode retornar nenhuma, uma ou várias linhas.

Quando escolher uma subconsulta de coluna

Use esse formato quando a consulta interna gerar uma lista de identificadores, preços, categorias ou outros valores que devem ser testados pela consulta principal.

6.1 Clientes que já realizaram compras

Código 21: Pertencimento com IN

```sql
SELECT
    id_cliente,
    nome,
    cidade
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM vendas
)
ORDER BY nome;
```

Leia o resultado

A consulta interna produz os identificadores presentes nas vendas. A externa mostra os clientes cujo identificador pertence a esse conjunto.

6.2 Produtos que já foram vendidos

Código 22: Produtos presentes na tabela vendas

```sql
SELECT
    id_produto,
    nome_produto,
    categoria
FROM produtos
WHERE id_produto IN (
    SELECT id_produto
    FROM vendas
)
ORDER BY nome_produto;
```

6.3 Clientes que pagaram com Pix

Código 23: Lista de clientes filtrada pela forma de pagamento

```sql
SELECT
    nome,
    cidade,
    estado
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM vendas
    WHERE forma_pagamento = 'Pix'
)
ORDER BY nome;
```

6.4 Produtos comprados por clientes mineiros

Código 24: Subconsultas aninhadas em dois níveis

```sql
SELECT
    nome_produto,
    categoria
FROM produtos
WHERE id_produto IN (
    SELECT id_produto
    FROM vendas
    WHERE id_cliente IN (
        SELECT id_cliente
        FROM clientes
        WHERE estado = 'MG'
    )
)
ORDER BY nome_produto;
```

Leia de dentro para fora

Primeiro identificamos clientes de MG. Depois identificamos produtos comprados por esses clientes. Por fim, consultamos o cadastro dos produtos encontrados.

6.5 DISTINCT é obrigatório?

Código 25: Removendo repetições explicitamente

```sql
SELECT nome
FROM clientes
WHERE id_cliente IN (
    SELECT DISTINCT id_cliente
    FROM vendas
);
```

`DISTINCT` não altera o significado lógico de `IN`: um cliente pertence ou não pertence ao conjunto. Pode, no entanto, ajudar a tornar a intenção explícita em exercícios introdutórios.

Atenção

Em `id_cliente IN (subconsulta)`, a subconsulta deve retornar somente uma coluna. Retornar `id_cliente, id_produto` sem usar uma tupla produz erro de incompatibilidade de colunas.

Atividade 6.1

Use `IN` para:

a) listar clientes que já compraram;

b) listar produtos que já foram vendidos;

c) listar clientes que compraram com Cartão;

d) listar produtos vendidos em abril de 2025;

e) listar clientes que compraram produtos da categoria Telefonia;

f) listar produtos comprados por clientes do estado de São Paulo.

7 NOT IN e o problema dos valores nulos

`NOT IN` seleciona valores que não aparecem no resultado da subconsulta.

7.1 Clientes sem compras

Código 26: Clientes ausentes da tabela vendas

```sql
SELECT
    id_cliente,
```

```
    nome,
    cidade
FROM clientes
WHERE id_cliente NOT IN (
    SELECT id_cliente
    FROM vendas
)
ORDER BY nome;
```

7.2 Produtos nunca vendidos

Código 27: Produtos ausentes das vendas

```
SELECT
    id_produto,
    nome_produto,
    estoque
FROM produtos
WHERE id_produto NOT IN (
    SELECT id_produto
    FROM vendas
)
ORDER BY nome_produto;
```

Leia o resultado

Nesse banco, `vendas.id_cliente` e `vendas.id_produto` são `NOT NULL`; por isso, os exemplos acima são seguros quanto a valores nulos na coluna retornada.

7.3 Por que um NULL pode invalidar o resultado?

Considere esta expressão:

Código 28: Lista contendo valor nulo

```
SELECT 4 NOT IN (1, 2, NULL);
```

O resultado é `NULL`, e não verdadeiro. O banco não consegue afirmar que 4 é diferente de um valor desconhecido.

Atenção

Se uma subconsulta usada em `NOT IN` retornar `NULL`, uma consulta que aparentemente deveria encontrar registros pode não retornar linha alguma. Essa é uma das armadilhas mais importantes das subconsultas SQL.

7.4 Primeira solução: eliminar os nulos

Código 29: Protegendo NOT IN contra NULL

```sql
SELECT nome
FROM clientes
WHERE id_cliente NOT IN (
    SELECT id_cliente
    FROM vendas
    WHERE id_cliente IS NOT NULL
);
```

7.5 Segunda solução: preferir NOT EXISTS

Código 30: Ausência de correspondência com NOT EXISTS

```sql
SELECT
    c.nome,
    c.cidade
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
)
ORDER BY c.nome;
```

Qual tipo devo usar?

Use `NOT IN` quando a subconsulta retorna uma coluna sem nulos e a ideia de "não pertencer à lista" é clara. Prefira `NOT EXISTS` quando a pergunta for "não existe linha relacionada" ou quando houver possibilidade de valores `NULL`.

Atividade 7.1

Responda e execute:

a) liste os clientes que nunca compraram usando `NOT IN`;

b) liste os produtos nunca vendidos usando `NOT IN`;

c) explique o efeito de `NULL` em `NOT IN`;

d) reescreva os itens a e b usando `NOT EXISTS`;

e) explique por que `id_produto` em vendas não pode ser nulo.

8 Comparações com ANY, SOME e ALL

Esses operadores permitem comparar um único valor com vários valores retornados por uma subconsulta de uma coluna.

Expressão	Interpretação
`> ANY (...)`	maior que pelo menos um dos valores;
`> ALL (...)`	maior que todos os valores;
`< ANY (...)`	menor que pelo menos um dos valores;
`< ALL (...)`	menor que todos os valores;
`= ANY (...)`	equivalente a `IN (...)` ;
`SOME`	sinônimo de `ANY`.

8.1 Maior que algum produto de Acessórios

Código 31: Comparação com pelo menos um valor

```sql
SELECT
    nome_produto,
    categoria,
    preco
FROM produtos
WHERE preco > ANY (
    SELECT preco
    FROM produtos
    WHERE categoria = 'Acessórios'
)
ORDER BY preco;
```

Se os preços de Acessórios forem 120, 210 e 350, a condição significa que o produto precisa ser maior que pelo menos um deles.

8.2 Maior que todos os produtos de Acessórios

Código 32: Comparação com todos os valores

```sql
SELECT
    nome_produto,
    categoria,
    preco
FROM produtos
WHERE preco > ALL (
    SELECT preco
    FROM produtos
    WHERE categoria = 'Acessórios'
)
ORDER BY preco;
```

Nesse caso, o preço deve ser maior que 120, maior que 210 e maior que 350.

8.3 Relação com MIN e MAX

Quando a subconsulta possui valores não nulos e não está vazia:

Expressão	Equivalência didática
`x > ANY(lista)`	`x > MIN(lista)`
`x > ALL(lista)`	`x > MAX(lista)`
`x < ANY(lista)`	`x < MAX(lista)`
`x < ALL(lista)`	`x < MIN(lista)`

Atenção

Essas equivalências precisam de cuidado quando a subconsulta está vazia ou contém `NULL`. Em particular, uma comparação com `ALL` sobre um conjunto vazio é verdadeira, enquanto `MAX` de um conjunto vazio retorna `NULL`.

Qual tipo devo usar?

Escolha `ANY` quando a pergunta disser "pelo menos um". Escolha `ALL` quando a pergunta disser "todos". Se a intenção for simplesmente comparar com um máximo ou mínimo conhecido, uma subconsulta escalar com `MAX` ou `MIN` costuma ser mais fácil de explicar.

Atividade 8.1

Escreva consultas para encontrar:

a) produtos mais caros que pelo menos um produto de Telefonia;

b) produtos mais caros que todos os produtos de Acessórios;

c) produtos mais baratos que todos os produtos de Escritório;

d) produtos cujo preço coincida com algum preço da categoria Informática;

e) uma versão com `MAX` equivalente ao item b, considerando o conjunto existente e sem valores nulos.

9 Subconsultas de linha: várias colunas em uma linha

Uma subconsulta de linha retorna **uma única linha com duas ou mais colunas**. A consulta externa compara uma **tupla** com outra tupla.

O que é uma tupla?

Uma tupla é um conjunto ordenado de valores, como `(categoria, preco)` ou `(id_cliente, id_produto)`. O número e a ordem das colunas precisam coincidir dos dois lados.

9.1 Mesma categoria e mesmo preço de um produto

Código 33: Comparação entre duas colunas

```sql
SELECT
    nome_produto,
    categoria,
    preco
FROM produtos
WHERE (categoria, preco) = (
    SELECT categoria, preco
    FROM produtos
    WHERE id_produto = 3
);
```

Leia o resultado

A subconsulta retorna uma linha com duas colunas: a categoria e o preço do produto 3. A consulta externa exige coincidência simultânea dessas duas informações.

9.2 Mesmo cliente e produto de uma venda

Código 34: Comparando um par de identificadores

```sql
SELECT
    id_venda,
    id_cliente,
    id_produto,
    data_venda
FROM vendas
WHERE (id_cliente, id_produto) = (
    SELECT id_cliente, id_produto
    FROM vendas
    WHERE id_venda = 1
);
```

9.3 O que acontece quando as quantidades não coincidem?

Código 35: Exemplo incorreto: uma coluna contra duas

```sql
SELECT nome_produto
FROM produtos
WHERE preco = (
    SELECT categoria, preco
    FROM produtos
    WHERE id_produto = 3
);
```

O MySQL pode informar:

```
ERROR 1241 (21000): Operand should contain 1 column(s)
```

Atenção

Se a esquerda tem uma coluna, a direita deve produzir uma coluna. Se a esquerda tem `(categoria, preco)`, a direita deve produzir duas colunas na mesma ordem. Além disso, com `=`, a subconsulta deve retornar no máximo uma linha.

Qual tipo devo usar?

Escolha subconsulta de linha quando precisar comparar várias características simultaneamente com uma **única linha de referência**.

Atividade 9.1

Use comparações de tuplas para:

a) encontrar produtos com a mesma categoria e preço do produto 3;

b) encontrar vendas com o mesmo cliente e produto da venda 1;

c) comparar cidade e estado com os dados do cliente 1;

d) explicar o erro de comparar uma coluna com uma subconsulta de duas colunas;

e) explicar por que `=` não aceita várias linhas nesse formato.

10 Subconsultas com várias linhas e várias colunas

Uma subconsulta pode retornar diversas linhas e diversas colunas. Nesse caso, ela representa uma **tabela intermediária**.

Há duas aplicações didáticas importantes:

1. comparação de tuplas com `IN`;
2. utilização como tabela derivada na cláusula `FROM`.

10.1 Tuplas com IN

Código 36: Produto mais caro de cada categoria

```sql
SELECT
    nome_produto,
    categoria,
    preco
FROM produtos
WHERE (categoria, preco) IN (
    SELECT
        categoria,
        MAX(preco)
    FROM produtos
    GROUP BY categoria
)
```

```
ORDER BY categoria;
```

Leia o resultado

A subconsulta retorna várias linhas e duas colunas. Cada linha contém uma categoria e seu maior preço. A consulta externa verifica se o par `(categoria, preco)` coincide com algum dos pares retornados.

categoria	MAX(preco)
Acessórios	350,00
Escritório	1350,00
Informática	4590,00
Telefonia	2850,00

10.2 Venda mais recente de cada cliente

Código 37: Par cliente e data mais recente

```
SELECT
    id_venda,
    id_cliente,
    data_venda,
    valor_unitario
FROM vendas
WHERE (id_cliente, data_venda) IN (
    SELECT
        id_cliente,
        MAX(data_venda)
    FROM vendas
    GROUP BY id_cliente
)
ORDER BY id_cliente;
```

Atenção

Se um cliente tiver duas vendas na mesma data máxima, as duas serão exibidas. Se for necessário escolher apenas uma, deve-se definir um critério adicional, como o maior `id_venda`.

Qual tipo devo usar?

Use `(coluna1, coluna2) IN (subconsulta)` quando a subconsulta retornar várias linhas e o teste exigir coincidência simultânea de várias colunas.

Atividade 10.1

Escreva consultas para:

a) encontrar o produto mais caro de cada categoria;

b) encontrar o produto mais barato de cada categoria;

c) encontrar a venda mais recente de cada cliente;

d) encontrar a venda de maior valor unitário de cada produto;

e) explicar quantas colunas a subconsulta deve retornar em cada caso.

11 Subconsultas no FROM: tabelas derivadas

Uma subconsulta no `FROM` transforma seu resultado em uma tabela intermediária que pode ser consultada pela instrução externa.

Quando escolher uma tabela derivada

Use uma subconsulta no `FROM` quando for necessário produzir um relatório intermediário e, depois, filtrar, ordenar ou agregar novamente esse relatório.

11.1 Total gasto por cliente

Código 38: Construindo uma tabela intermediária

```sql
SELECT
    resumo.id_cliente,
    resumo.total_gasto
FROM (
    SELECT
        id_cliente,
        SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo
ORDER BY resumo.total_gasto DESC;
```

O alias da tabela derivada

No MySQL, a tabela derivada deve receber um nome, como `AS resumo`. Sem esse alias, a consulta não é aceita.

11.2 Filtrando um relatório intermediário

Código 39: Clientes com gasto total acima de 1000

```sql
SELECT
    resumo.id_cliente,
    resumo.total_gasto
FROM (
    SELECT
        id_cliente,
```

```
        SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo
WHERE resumo.total_gasto > 1000
ORDER BY resumo.total_gasto DESC;
```

11.3 Calculando a média dos totais por cliente

Código 40: Agregação sobre um resultado já agregado

```
SELECT
    AVG(resumo.total_gasto) AS media_gasta_por_cliente
FROM (
    SELECT
        id_cliente,
        SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo;
```

Leia o resultado

Primeiro calculamos o total individual de cada cliente. Depois calculamos a média entre esses totais. Isso é diferente de calcular a média de todas as vendas individualmente.

11.4 Juntando a tabela derivada com clientes

Código 41: Nome do cliente e total gasto

```
SELECT
    c.nome,
    c.cidade,
    resumo.total_gasto
FROM clientes AS c
INNER JOIN (
    SELECT
        id_cliente,
        SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo
    ON c.id_cliente = resumo.id_cliente
ORDER BY resumo.total_gasto DESC;
```

11.5 Incluindo clientes sem compras

Código 42: Tabela derivada combinada com LEFT JOIN

```sql
SELECT
    c.nome,
    COALESCE(resumo.total_gasto, 0) AS total_gasto
FROM clientes AS c
LEFT JOIN (
    SELECT
        id_cliente,
        SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo
    ON c.id_cliente = resumo.id_cliente
ORDER BY total_gasto DESC, c.nome;
```

Atenção

Uma tabela derivada pode ter várias linhas e várias colunas. Não tente usá-la diretamente com `preco = (subconsulta)`: a posição do `FROM` é que permite tratá-la como tabela.

Atividade 11.1

Use subconsultas no `FROM` para:

a) calcular o total gasto por cliente;

b) calcular a média dos totais gastos pelos clientes que compraram;

c) listar apenas totais superiores a 1.500;

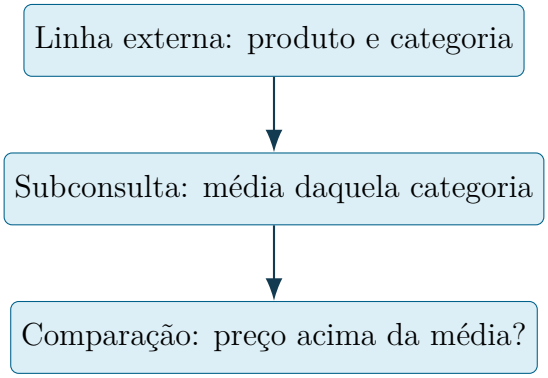
d) mostrar o nome do cliente junto de seu total gasto;

e) incluir clientes sem compras e exibir zero;

f) calcular a média do faturamento total por categoria.

12 Subconsultas correlacionadas

Uma subconsulta correlacionada utiliza alguma coluna da consulta externa. Por isso, sua pergunta muda de acordo com a linha externa analisada.



12.1 Produtos acima da média de sua própria categoria

Código 43: Correlação pela coluna categoria

```sql
SELECT
    p1.nome_produto,
    p1.categoria,
    p1.preco
FROM produtos AS p1
WHERE p1.preco > (
    SELECT AVG(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
)
ORDER BY p1.categoria, p1.preco DESC;
```

Leia o resultado

Para um produto de Telefonia, a subconsulta calcula a média de Telefonia. Para um produto de Informática, calcula a média de Informática. A linha externa define qual conjunto interno será analisado.

12.2 Produto mais caro de cada categoria

Código 44: Máximo dentro da categoria da linha externa

```sql
SELECT
    p1.nome_produto,
    p1.categoria,
    p1.preco
FROM produtos AS p1
WHERE p1.preco = (
    SELECT MAX(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
)
ORDER BY p1.categoria;
```

12.3 Venda mais recente de cada cliente

Código 45: Correlação por identificador de cliente

```sql
SELECT
    v1.id_venda,
    v1.id_cliente,
    v1.data_venda
FROM vendas AS v1
WHERE v1.data_venda = (
    SELECT MAX(v2.data_venda)
    FROM vendas AS v2
    WHERE v2.id_cliente = v1.id_cliente
)
ORDER BY v1.id_cliente;
```

12.4 Venda acima da média do próprio produto

Código 46: Comparação com a média histórica de cada produto

```sql
SELECT
    v1.id_venda,
    v1.id_produto,
    v1.valor_unitario
FROM vendas AS v1
WHERE v1.valor_unitario > (
    SELECT AVG(v2.valor_unitario)
    FROM vendas AS v2
    WHERE v2.id_produto = v1.id_produto
)
ORDER BY v1.id_produto, v1.valor_unitario;
```

Qual tipo devo usar?

Use uma subconsulta correlacionada quando a condição interna depender de informações da linha externa: “da mesma categoria”, “do mesmo cliente”, “do mesmo produto” ou “deste estado”.

Atividade 12.1

Crie subconsultas correlacionadas para:

a) encontrar produtos acima da média da própria categoria;

b) encontrar o produto mais barato de cada categoria;

c) encontrar a venda mais recente de cada cliente;

d) encontrar vendas acima da média de preço do mesmo produto;

e) encontrar produtos com margem superior à margem média da própria categoria.

13 EXISTS e NOT EXISTS

`EXISTS` não pergunta qual valor foi retornado. Ele pergunta somente se a subconsulta produziu pelo menos uma linha.

A pergunta correta

`EXISTS` significa: existe pelo menos uma linha que satisfaz a condição? `NOT EXISTS` significa: não existe nenhuma linha que satisfaz a condição?

13.1 Clientes que já compraram

Código 47: Existência de vendas para cada cliente

```sql
SELECT
    c.id_cliente,
    c.nome
FROM clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
)
ORDER BY c.nome;
```

13.2 Clientes que nunca compraram

Código 48: Ausência de vendas relacionadas

```sql
SELECT
    c.id_cliente,
    c.nome
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
)
ORDER BY c.nome;
```

13.3 Produtos nunca vendidos

Código 49: Ausência de vendas para cada produto

```sql
SELECT
    p.nome_produto,
    p.categoria,
    p.estoque
FROM produtos AS p
```

```sql
WHERE NOT EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_produto = p.id_produto
)
ORDER BY p.nome_produto;
```

13.4 Clientes que compraram produtos de Telefonia

Código 50: EXISTS com junção dentro da subconsulta

```sql
SELECT
    c.nome,
    c.cidade
FROM clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM vendas AS v
    INNER JOIN produtos AS p
        ON v.id_produto = p.id_produto
    WHERE v.id_cliente = c.id_cliente
      AND p.categoria = 'Telefonia'
)
ORDER BY c.nome;
```

13.5 Por que escrever SELECT 1?

Código 51: A coluna selecionada não altera o teste de existência

```sql
WHERE EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
)
```

`SELECT 1` comunica que o conteúdo das colunas não interessa. O que importa é existir pelo menos uma linha correspondente. O teste também poderia usar outra expressão, mas `SELECT 1` é uma convenção legível.

Qual tipo devo usar?

Use `EXISTS` quando a pergunta for "existe pelo menos uma correspondência?". Use `NOT EXISTS` quando a pergunta for "não existe correspondência?". Essa escolha costuma representar melhor a intenção do que contar linhas somente para verificar presença.

Atividade 13.1

Use `EXISTS` ou `NOT EXISTS` para:

a) encontrar clientes que compraram;

b) encontrar clientes que nunca compraram;

c) encontrar produtos que nunca foram vendidos;

d) encontrar clientes que pagaram alguma compra por Pix;

e) encontrar clientes que compraram algum produto de Informática;

f) encontrar produtos que nunca foram comprados por clientes de MG.

14 Subconsultas na lista do SELECT

Uma subconsulta usada na lista do `SELECT` precisa retornar um único valor para cada linha externa.

Quando escolher uma subconsulta no SELECT

Use esse formato quando precisar acrescentar uma informação calculada à linha externa, como quantidade de compras, total gasto ou data da última venda.

14.1 Quantidade de compras por cliente

Código 52: Contagem correlacionada como coluna calculada

```sql
SELECT
    c.nome,
    (
        SELECT COUNT(*)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente
    ) AS quantidade_compras
FROM clientes AS c
ORDER BY quantidade_compras DESC, c.nome;
```

Leia o resultado

Todos os clientes aparecem. Para Daniel, Gabriela e Karen, a subconsulta com `COUNT(*)` retorna zero.

14.2 Total gasto por cliente

Código 53: Total individual preservando clientes sem compra

```sql
SELECT
    c.nome,
    COALESCE((
        SELECT SUM(v.quantidade * v.valor_unitario)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente
```

```
    ), 0) AS total_gasto
FROM clientes AS c
ORDER BY total_gasto DESC, c.nome;
```

14.3 Data da última compra

Código 54: Data máxima para cada cliente

```
SELECT
    c.nome,
    (
        SELECT MAX(v.data_venda)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente
    ) AS ultima_compra
FROM clientes AS c
ORDER BY c.nome;
```

14.4 Diferença entre COUNT e SUM em conjunto vazio

Função	Sem linhas correspondentes	Tratamento
`COUNT(*)`	retorna 0	normalmente dispensa `COALESCE`;
`SUM(...)`	retorna `NULL`	usar `COALESCE(..., 0)`;
`AVG(...)`	retorna `NULL`	definir se zero faz sentido;
`MAX(...)`	retorna `NULL`	interpretar como ausência de valor.

Atenção

Não escreva, na lista do `SELECT`, uma subconsulta que possa devolver várias linhas. Cada coluna de uma linha externa precisa receber no máximo um valor escalar.

Atividade 14.1

Acrescente ao resultado de cada cliente:

a) a quantidade total de compras;

b) o total gasto;

c) a data da compra mais recente;

d) o maior valor unitário já pago;

e) a quantidade de compras pagas por Pix.

15 Subconsultas no HAVING

A cláusula `HAVING` filtra grupos já formados por `GROUP BY`. Uma subconsulta permite comparar o resultado agregado com outro valor calculado.

15.1 Categorias acima da média dos preços gerais

Código 55: Média por categoria acima da média geral

```sql
SELECT
    categoria,
    AVG(preco) AS media_categoria
FROM produtos
GROUP BY categoria
HAVING AVG(preco) > (
    SELECT AVG(preco)
    FROM produtos
)
ORDER BY media_categoria DESC;
```

15.2 Clientes com gasto acima da média dos totais

Código 56: Comparando grupos com a média de grupos

```sql
SELECT
    c.id_cliente,
    c.nome,
    SUM(v.quantidade * v.valor_unitario) AS total_gasto
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING SUM(v.quantidade * v.valor_unitario) > (
    SELECT AVG(resumo.total_gasto)
    FROM (
        SELECT
            id_cliente,
            SUM(quantidade * valor_unitario) AS total_gasto
        FROM vendas
        GROUP BY id_cliente
    ) AS resumo
)
ORDER BY total_gasto DESC;
```

Leia o resultado

Primeiro calculamos o total de cada cliente. Depois obtemos a média desses totais. Por fim, mantemos apenas clientes cujo gasto supera essa média.

15.3 Categorias com faturamento acima do faturamento médio

Código 57: Comparação entre faturamentos de categorias

```sql
SELECT
    p.categoria,
    SUM(v.quantidade * v.valor_unitario) AS faturamento
FROM produtos AS p
INNER JOIN vendas AS v
    ON p.id_produto = v.id_produto
GROUP BY p.categoria
HAVING SUM(v.quantidade * v.valor_unitario) > (
    SELECT AVG(totais.faturamento)
    FROM (
        SELECT
            p2.categoria,
            SUM(v2.quantidade * v2.valor_unitario) AS faturamento
        FROM produtos AS p2
        INNER JOIN vendas AS v2
            ON p2.id_produto = v2.id_produto
        GROUP BY p2.categoria
    ) AS totais
)
ORDER BY faturamento DESC;
```

Qual tipo devo usar?

Escolha `HAVING` quando a comparação envolver valores produzidos por `GROUP BY`, como total gasto por cliente ou faturamento por categoria. Escolha `WHERE` quando estiver filtrando linhas individuais antes do agrupamento.

Atividade 15.1

Use `HAVING` e subconsultas para:

a) encontrar categorias cuja média de preços supera a média geral;

b) encontrar clientes com gasto total acima da média dos totais;

c) encontrar produtos cuja quantidade vendida supera a média das quantidades totais vendidas por produto;

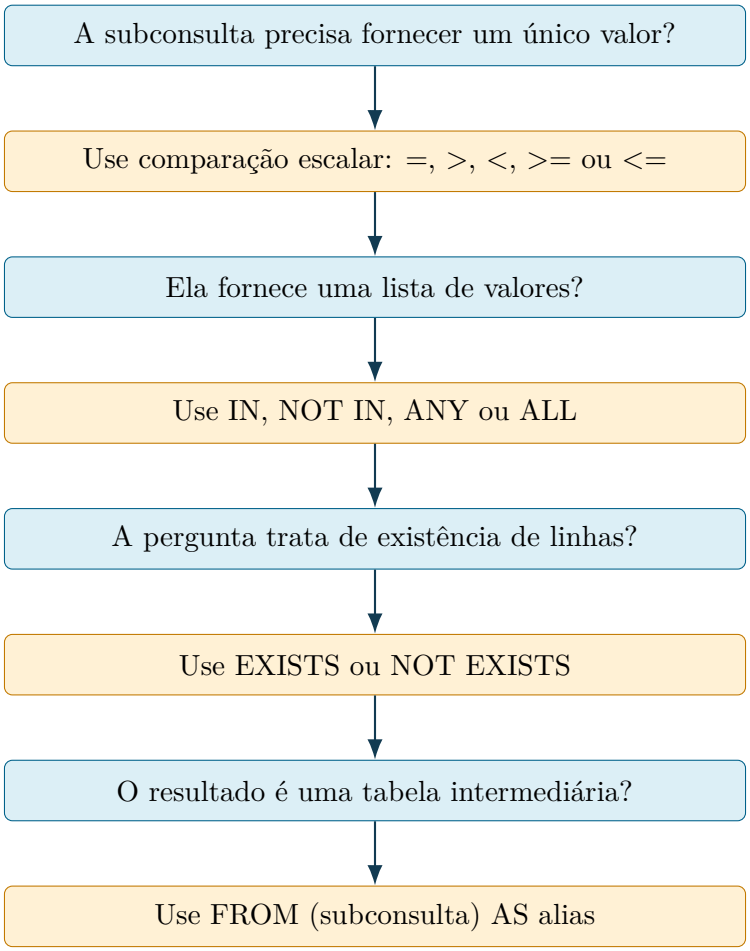
d) explicar por que a comparação do total agrupado não deve ser feita no `WHERE`.

16 Como escolher a subconsulta correta

A escolha começa pela pergunta de negócio. Em seguida, identificamos o formato do resultado intermediário e o operador compatível.

Pergunta	Formato	Estrutura adequada
Maior que a média?	1 valor	`> (SELECT AVG(...))`
Está na lista?	1 coluna	`IN (SELECT id ...)`
Não está na lista?	1 coluna	`NOT IN`, se não houver `NULL`
Existe correspondência?	existência	`EXISTS (...)`
Não existe correspondência?	existência	`NOT EXISTS (...)`
Maior que pelo menos um?	1 coluna	`> ANY (...)`
Maior que todos?	1 coluna	`> ALL (...)`
Mesmo par de valores?	1 linha	`(a,b) = (SELECT x,y ...)`
Par presente em vários pares?	tabela	`(a,b) IN (SELECT x,y ...)`
Consultar um resumo intermediário?	tabela	`FROM (SELECT ...) AS t`
Depende da linha atual?	variável	subconsulta correlacionada

16.1 Fluxo de decisão



16.2 O mesmo problema visto por abordagens diferentes

Pergunta: quais clientes realizaram alguma compra?

Código 58: Abordagem por lista de identificadores

```
SELECT nome
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM vendas
);
```

Código 59: Abordagem por existência de correspondência

```
SELECT c.nome
FROM clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
);
```

Código 60: Abordagem por junção e eliminação de repetições

```
SELECT DISTINCT c.nome
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

Conceito-chave

As três consultas respondem à mesma pergunta, mas destacam ideias diferentes: pertencimento, existência e relacionamento entre tabelas.

17 Erros frequentes

17.1 Erro 1: várias linhas onde se espera uma

Código 61: Comparação escalar recebendo vários preços

```
SELECT nome_produto
FROM produtos
WHERE preco = (
    SELECT preco
    FROM produtos
    WHERE categoria = 'Informática'
);
```

```
ERROR 1242 (21000): Subquery returns more than 1 row
```

Correções possíveis, dependendo da intenção:

Código 62: Comparação com uma lista

```
WHERE preco IN (
    SELECT preco
    FROM produtos
    WHERE categoria = 'Informática'
)
```

Código 63: Comparação com um único máximo

```
WHERE preco = (
    SELECT MAX(preco)
    FROM produtos
    WHERE categoria = 'Informática'
)
```

17.2 Erro 2: várias colunas onde se espera uma

Código 64: Subconsulta incompatível com o operador

```
SELECT nome_produto
FROM produtos
WHERE id_produto IN (
    SELECT id_produto, quantidade
    FROM vendas
);
```

```
ERROR 1241 (21000): Operand should contain 1 column(s)
```

Se a comparação envolver somente `id_produto`, a subconsulta deve selecionar somente `id_produto`.

17.3 Erro 3: esquecer o alias de uma tabela derivada

Código 65: Consulta incorreta: falta o alias

```
SELECT total_gasto
FROM (
    SELECT SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
);
```

```
ERROR 1248 (42000): Every derived table must have its own alias
```

Código 66: Versão corrigida

```
SELECT resumo.total_gasto
FROM (
```

```
    SELECT SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
) AS resumo;
```

17.4 Erro 4: usar NOT IN com valores nulos

Código 67: Resultado desconhecido devido a NULL

```
SELECT 4 NOT IN (1, 2, NULL);
```

A alternativa geralmente mais robusta para testar ausência de relação é `NOT EXISTS`.

17.5 Erro 5: usar LIMIT 1 sem critério

Código 68: Resultado arbitrário sem ORDER BY

```
SELECT nome_produto
FROM produtos
WHERE preco > (
    SELECT preco
    FROM produtos
    WHERE categoria = 'Informática'
    LIMIT 1
);
```

Sem `ORDER BY`, não está claro qual produto da categoria servirá como referência.

Código 69: Referência definida explicitamente

```
SELECT nome_produto
FROM produtos
WHERE preco > (
    SELECT preco
    FROM produtos
    WHERE categoria = 'Informática'
    ORDER BY preco DESC, id_produto ASC
    LIMIT 1
);
```

Se a intenção é obter o maior preço, uma alternativa mais direta é usar `MAX(preco)`.

17.6 Erro 6: esquecer a correlação

Código 70: Média geral, e não média da categoria

```
SELECT p1.nome_produto, p1.categoria, p1.preco
FROM produtos AS p1
WHERE p1.preco > (
    SELECT AVG(p2.preco)
    FROM produtos AS p2
);
```

Para obter a média da categoria de cada produto:

Código 71: Correlação incluída corretamente

```
WHERE p1.preco > (
    SELECT AVG(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
)
```

17.7 Erro 7: confundir média de vendas com média de clientes

Código 72: Média de cada venda individual

```
SELECT AVG(quantidade * valor_unitario)
FROM vendas;
```

Código 73: Média dos totais gastos por cliente

```
SELECT AVG(resumo.total_gasto)
FROM (
    SELECT
        id_cliente,
        SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo;
```

São perguntas diferentes. A primeira trata cada venda como unidade de análise; a segunda trata cada cliente que comprou como unidade de análise.

Atividade 17.1

Para cada erro desta seção, escreva:

a) qual formato a consulta externa esperava;

b) qual formato a subconsulta realmente produziu;

c) qual alteração corrige o problema;

d) se a correção modifica o significado da pergunta.

18 Subconsultas e junções: comparação

Subconsultas e junções não são rivais. Em muitos problemas, ambas podem produzir resultados equivalentes. A melhor escolha depende da pergunta, da clareza e do plano de execução.

18.1 Clientes que compraram

Código 74: Usando subconsulta com IN

```sql
SELECT nome
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM vendas
);
```

Código 75: Usando INNER JOIN

```sql
SELECT DISTINCT c.nome
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

18.2 Clientes que nunca compraram

Código 76: Usando NOT EXISTS

```sql
SELECT c.nome
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
);
```

Código 77: Usando LEFT JOIN e IS NULL

```sql
SELECT c.nome
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
WHERE v.id_venda IS NULL;
```

18.3 Produto mais caro por categoria

Código 78: Usando subconsulta correlacionada

```sql
SELECT p1.nome_produto, p1.categoria, p1.preco
FROM produtos AS p1
WHERE p1.preco = (
    SELECT MAX(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);
```

Código 79: Usando junção com tabela derivada

```sql
SELECT
    p.nome_produto,
    p.categoria,
    p.preco
FROM produtos AS p
INNER JOIN (
    SELECT
        categoria,
        MAX(preco) AS maior_preco
    FROM produtos
    GROUP BY categoria
) AS maximos
    ON p.categoria = maximos.categoria
   AND p.preco = maximos.maior_preco;
```

Desempenho e legibilidade

Uma subconsulta correlacionada pode ser muito clara didaticamente, mas sua execução merece avaliação em tabelas grandes. O MySQL pode otimizar muitas consultas automaticamente. A comparação real deve ser feita com índices, volume representativo e ferramentas como `EXPLAIN`.

19 Revisão geral: estudo dirigido

Antes de resolver cada questão, escreva ao lado qual formato de subconsulta pretende utilizar: escalar, coluna, linha, tabela ou correlacionada.

Atividade 19.1 — classificação

1. Classifique `SELECT MAX(preco) FROM produtos`.
2. Classifique `SELECT id_cliente FROM vendas`.
3. Classifique `SELECT categoria, preco FROM produtos WHERE id_produto = 1`.
4. Classifique um resumo por cliente com identificador e total gasto.
5. Identifique o operador apropriado para testar pertencimento a uma coluna.
6. Identifique a estrutura apropriada para testar existência.
7. Identifique a estrutura apropriada para consultar um resumo intermediário.

Atividade 19.2 — subconsultas escalares

1. Liste produtos acima do preço médio.
2. Encontre o produto mais barato.
3. Liste vendas realizadas na data mais recente.

4. Liste produtos com custo acima do custo médio.
5. Liste vendas com valor total acima da média dos valores das vendas.

Atividade 19.3 — listas, presença e ausência

1. Liste clientes que compraram usando `IN`.
2. Liste clientes que nunca compraram usando `NOT EXISTS`.
3. Liste produtos nunca vendidos.
4. Liste clientes que compraram produtos de Telefonia.
5. Liste produtos comprados por clientes de MG.
6. Explique por que `NOT IN` exige atenção quando há `NULL`.

Atividade 19.4 — correlação e agrupamentos

1. Encontre o produto mais caro de cada categoria.
2. Encontre produtos acima da média da própria categoria.
3. Encontre a venda mais recente de cada cliente.
4. Mostre a quantidade de compras ao lado do nome de cada cliente.
5. Mostre o total gasto ao lado do nome de cada cliente.
6. Encontre clientes com gasto acima da média dos totais por cliente.
7. Compare a solução por subconsulta com uma solução usando junção.

Atividade 19.5 — desafios integradores

1. Liste os produtos com margem superior à margem média de sua categoria.
2. Liste os clientes que compraram algum produto acima do preço médio geral.
3. Liste os produtos nunca comprados por clientes do estado de MG.
4. Mostre categorias com faturamento acima da média dos faturamentos.
5. Liste a última venda de cada cliente, exibindo também seu nome.
6. Identifique clientes que gastaram mais do que todos os clientes de SP.

20 Gabarito sugerido

Mais de uma solução pode estar correta. O mais importante é justificar o formato retornado e o operador escolhido.

20.1 Atividade 4.1

a) Escalar: uma linha e uma coluna.

b) Coluna: várias linhas e uma coluna.

c) Linha: uma linha e duas colunas, pois a chave primária é única.

d) Tabela: várias linhas e duas colunas.

e) Escalar: a função `AVG` retorna um valor agregado.

20.2 Atividade 5.1

```sql
SELECT nome_produto, preco
FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos);

SELECT nome_produto, preco
FROM produtos
WHERE preco = (SELECT MIN(preco) FROM produtos);

SELECT id_venda, data_venda
FROM vendas
WHERE data_venda = (SELECT MAX(data_venda) FROM vendas);

SELECT nome_produto, custo_aquisicao
FROM produtos
WHERE custo_aquisicao > (
    SELECT AVG(custo_aquisicao)
    FROM produtos
);

SELECT nome_produto, preco - custo_aquisicao AS margem
FROM produtos
WHERE preco - custo_aquisicao > (
    SELECT AVG(preco - custo_aquisicao)
    FROM produtos
);
```

20.3 Atividade 6.1

```sql
SELECT nome
FROM clientes
WHERE id_cliente IN (SELECT id_cliente FROM vendas);

SELECT nome_produto
FROM produtos
WHERE id_produto IN (SELECT id_produto FROM vendas);

```

```sql
SELECT nome
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM vendas
    WHERE forma_pagamento = 'Cartão'
);

SELECT nome_produto
FROM produtos
WHERE id_produto IN (
    SELECT id_produto
    FROM vendas
    WHERE data_venda BETWEEN '2025-04-01' AND '2025-04-30'
);

SELECT nome
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM vendas
    WHERE id_produto IN (
        SELECT id_produto
        FROM produtos
        WHERE categoria = 'Telefonia'
    )
);

SELECT nome_produto
FROM produtos
WHERE id_produto IN (
    SELECT id_produto
    FROM vendas
    WHERE id_cliente IN (
        SELECT id_cliente
        FROM clientes
        WHERE estado = 'SP'
    )
);
```

20.4 Atividade 7.1

```sql
SELECT nome
FROM clientes
WHERE id_cliente NOT IN (SELECT id_cliente FROM vendas);

SELECT nome_produto
FROM produtos
WHERE id_produto NOT IN (SELECT id_produto FROM vendas);

SELECT c.nome
```

```
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
);

SELECT p.nome_produto
FROM produtos AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM vendas AS v
    WHERE v.id_produto = p.id_produto
);
```

Um valor `NULL` torna desconhecidas certas comparações com `NOT IN`. No modelo proposto, as chaves estrangeiras de vendas são declaradas `NOT NULL`.

20.5 Atividade 8.1

```
SELECT nome_produto, preco
FROM produtos
WHERE preco > ANY (
    SELECT preco FROM produtos WHERE categoria = 'Telefonia'
);

SELECT nome_produto, preco
FROM produtos
WHERE preco > ALL (
    SELECT preco FROM produtos WHERE categoria = 'Acessórios'
);

SELECT nome_produto, preco
FROM produtos
WHERE preco < ALL (
    SELECT preco FROM produtos WHERE categoria = 'Escritório'
);

SELECT nome_produto, preco
FROM produtos
WHERE preco = ANY (
    SELECT preco FROM produtos WHERE categoria = 'Informática'
);

SELECT nome_produto, preco
FROM produtos
WHERE preco > (
    SELECT MAX(preco)
    FROM produtos
    WHERE categoria = 'Acessórios'
);
```

20.6 Atividade 9.1

```sql
SELECT nome_produto, categoria, preco
FROM produtos
WHERE (categoria, preco) = (
    SELECT categoria, preco
    FROM produtos
    WHERE id_produto = 3
);

SELECT id_venda, id_cliente, id_produto
FROM vendas
WHERE (id_cliente, id_produto) = (
    SELECT id_cliente, id_produto
    FROM vendas
    WHERE id_venda = 1
);

SELECT nome, cidade, estado
FROM clientes
WHERE (cidade, estado) = (
    SELECT cidade, estado
    FROM clientes
    WHERE id_cliente = 1
);
```

Os lados de uma comparação devem possuir o mesmo número de colunas. Com =, a subconsulta de linha também deve retornar no máximo uma linha.

20.7 Atividade 10.1

```sql
SELECT nome_produto, categoria, preco
FROM produtos
WHERE (categoria, preco) IN (
    SELECT categoria, MAX(preco)
    FROM produtos
    GROUP BY categoria
);

SELECT nome_produto, categoria, preco
FROM produtos
WHERE (categoria, preco) IN (
    SELECT categoria, MIN(preco)
    FROM produtos
    GROUP BY categoria
);

SELECT id_venda, id_cliente, data_venda
FROM vendas
WHERE (id_cliente, data_venda) IN (
    SELECT id_cliente, MAX(data_venda)
```

```
    FROM vendas
    GROUP BY id_cliente
);

SELECT id_venda, id_produto, valor_unitario
FROM vendas
WHERE (id_produto, valor_unitario) IN (
    SELECT id_produto, MAX(valor_unitario)
    FROM vendas
    GROUP BY id_produto
);
```

20.8 Atividade 11.1

```
SELECT resumo.id_cliente, resumo.total_gasto
FROM (
    SELECT id_cliente,
           SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo;

SELECT AVG(resumo.total_gasto) AS media
FROM (
    SELECT id_cliente,
           SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo;

SELECT resumo.id_cliente, resumo.total_gasto
FROM (
    SELECT id_cliente,
           SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo
WHERE resumo.total_gasto > 1500;

SELECT c.nome, resumo.total_gasto
FROM clientes AS c
INNER JOIN (
    SELECT id_cliente,
           SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo ON c.id_cliente = resumo.id_cliente;

SELECT c.nome, COALESCE(resumo.total_gasto, 0) AS total_gasto
FROM clientes AS c
LEFT JOIN (
```

```
    SELECT id_cliente,
           SUM(quantidade * valor_unitario) AS total_gasto
    FROM vendas
    GROUP BY id_cliente
) AS resumo ON c.id_cliente = resumo.id_cliente;

SELECT AVG(totais.faturamento) AS media_categorias
FROM (
    SELECT p.categoria,
           SUM(v.quantidade * v.valor_unitario) AS faturamento
    FROM produtos AS p
    INNER JOIN vendas AS v ON p.id_produto = v.id_produto
    GROUP BY p.categoria
) AS totais;
```

20.9 Atividade 12.1

```
SELECT p1.nome_produto, p1.categoria, p1.preco
FROM produtos AS p1
WHERE p1.preco > (
    SELECT AVG(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);

SELECT p1.nome_produto, p1.categoria, p1.preco
FROM produtos AS p1
WHERE p1.preco = (
    SELECT MIN(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);

SELECT v1.id_venda, v1.id_cliente, v1.data_venda
FROM vendas AS v1
WHERE v1.data_venda = (
    SELECT MAX(v2.data_venda)
    FROM vendas AS v2
    WHERE v2.id_cliente = v1.id_cliente
);

SELECT v1.id_venda, v1.id_produto, v1.valor_unitario
FROM vendas AS v1
WHERE v1.valor_unitario > (
    SELECT AVG(v2.valor_unitario)
    FROM vendas AS v2
    WHERE v2.id_produto = v1.id_produto
);

SELECT p1.nome_produto, p1.categoria,
       p1.preco - p1.custo_aquisicao AS margem
```

```sql
FROM produtos AS p1
WHERE p1.preco - p1.custo_aquisicao > (
    SELECT AVG(p2.preco - p2.custo_aquisicao)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);
```

20.10 Atividade 13.1

```sql
SELECT c.nome
FROM clientes AS c
WHERE EXISTS (
    SELECT 1 FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
);

SELECT c.nome
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1 FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
);

SELECT p.nome_produto
FROM produtos AS p
WHERE NOT EXISTS (
    SELECT 1 FROM vendas AS v
    WHERE v.id_produto = p.id_produto
);

SELECT c.nome
FROM clientes AS c
WHERE EXISTS (
    SELECT 1 FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
      AND v.forma_pagamento = 'Pix'
);

SELECT c.nome
FROM clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM vendas AS v
    INNER JOIN produtos AS p ON v.id_produto = p.id_produto
    WHERE v.id_cliente = c.id_cliente
      AND p.categoria = 'Informática'
);

SELECT p.nome_produto
FROM produtos AS p
WHERE NOT EXISTS (
```

```
    SELECT 1
    FROM vendas AS v
    INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
    WHERE v.id_produto = p.id_produto
      AND c.estado = 'MG'
);
```

20.11 Atividade 14.1

```
SELECT c.nome,
       (SELECT COUNT(*)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente) AS compras
FROM clientes AS c;

SELECT c.nome,
       COALESCE((SELECT SUM(v.quantidade * v.valor_unitario)
                 FROM vendas AS v
                 WHERE v.id_cliente = c.id_cliente), 0) AS total
FROM clientes AS c;

SELECT c.nome,
       (SELECT MAX(v.data_venda)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente) AS ultima_compra
FROM clientes AS c;

SELECT c.nome,
       (SELECT MAX(v.valor_unitario)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente) AS maior_preco
FROM clientes AS c;

SELECT c.nome,
       (SELECT COUNT(*)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente
          AND v.forma_pagamento = 'Pix') AS compras_pix
FROM clientes AS c;
```

20.12 Atividade 15.1

```
SELECT categoria, AVG(preco) AS media_categoria
FROM produtos
GROUP BY categoria
HAVING AVG(preco) > (
    SELECT AVG(preco) FROM produtos
);

```

```sql
SELECT c.id_cliente, c.nome,
       SUM(v.quantidade * v.valor_unitario) AS total_gasto
FROM clientes AS c
INNER JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING SUM(v.quantidade * v.valor_unitario) > (
    SELECT AVG(resumo.total_gasto)
    FROM (
        SELECT id_cliente,
               SUM(quantidade * valor_unitario) AS total_gasto
        FROM vendas
        GROUP BY id_cliente
    ) AS resumo
);

SELECT p.id_produto, p.nome_produto,
       SUM(v.quantidade) AS unidades
FROM produtos AS p
INNER JOIN vendas AS v ON p.id_produto = v.id_produto
GROUP BY p.id_produto, p.nome_produto
HAVING SUM(v.quantidade) > (
    SELECT AVG(resumo.unidades)
    FROM (
        SELECT id_produto, SUM(quantidade) AS unidades
        FROM vendas
        GROUP BY id_produto
    ) AS resumo
);
```

20.13 Revisão 19.1 — classificação

1. Escalar: uma linha e uma coluna.
2. Coluna: várias linhas e uma coluna.
3. Linha: uma linha e duas colunas.
4. Tabela: várias linhas e duas colunas.
5. O operador apropriado é `IN`.
6. A estrutura apropriada é `EXISTS`.
7. A estrutura apropriada é `FROM (subconsulta) AS alias`.

20.14 Revisão 19.2 — subconsultas escalares

```sql
SELECT nome_produto, preco
FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos);

```

```sql
SELECT nome_produto, preco
FROM produtos
WHERE preco = (SELECT MIN(preco) FROM produtos);

SELECT id_venda, data_venda
FROM vendas
WHERE data_venda = (SELECT MAX(data_venda) FROM vendas);

SELECT nome_produto, custo_aquisicao
FROM produtos
WHERE custo_aquisicao > (
    SELECT AVG(custo_aquisicao) FROM produtos
);

SELECT id_venda, quantidade * valor_unitario AS total
FROM vendas
WHERE quantidade * valor_unitario > (
    SELECT AVG(quantidade * valor_unitario) FROM vendas
);
```

20.15 Revisão 19.3 — listas, presença e ausência

```sql
SELECT nome
FROM clientes
WHERE id_cliente IN (SELECT id_cliente FROM vendas);

SELECT c.nome
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1 FROM vendas AS v
    WHERE v.id_cliente = c.id_cliente
);

SELECT p.nome_produto
FROM produtos AS p
WHERE NOT EXISTS (
    SELECT 1 FROM vendas AS v
    WHERE v.id_produto = p.id_produto
);

SELECT c.nome
FROM clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM vendas AS v
    INNER JOIN produtos AS p ON v.id_produto = p.id_produto
    WHERE v.id_cliente = c.id_cliente
      AND p.categoria = 'Telefonia'
);

SELECT nome_produto
```

```
FROM produtos
WHERE id_produto IN (
    SELECT id_produto
    FROM vendas
    WHERE id_cliente IN (
        SELECT id_cliente FROM clientes WHERE estado = 'MG'
    )
);
```

20.16 Revisão 19.4 — correlação e agrupamentos

```
SELECT p1.nome_produto, p1.categoria, p1.preco
FROM produtos AS p1
WHERE p1.preco = (
    SELECT MAX(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);

SELECT p1.nome_produto, p1.categoria, p1.preco
FROM produtos AS p1
WHERE p1.preco > (
    SELECT AVG(p2.preco)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);

SELECT v1.id_venda, v1.id_cliente, v1.data_venda
FROM vendas AS v1
WHERE v1.data_venda = (
    SELECT MAX(v2.data_venda)
    FROM vendas AS v2
    WHERE v2.id_cliente = v1.id_cliente
);

SELECT c.nome,
       (SELECT COUNT(*)
        FROM vendas AS v
        WHERE v.id_cliente = c.id_cliente) AS compras
FROM clientes AS c;

SELECT c.nome,
       COALESCE((SELECT SUM(v.quantidade * v.valor_unitario)
                 FROM vendas AS v
                 WHERE v.id_cliente = c.id_cliente), 0) AS total
FROM clientes AS c;

SELECT c.id_cliente, c.nome,
       SUM(v.quantidade * v.valor_unitario) AS total
FROM clientes AS c
INNER JOIN vendas AS v ON c.id_cliente = v.id_cliente
```

```sql
GROUP BY c.id_cliente, c.nome
HAVING SUM(v.quantidade * v.valor_unitario) > (
    SELECT AVG(resumo.total)
    FROM (
        SELECT id_cliente,
               SUM(quantidade * valor_unitario) AS total
        FROM vendas
        GROUP BY id_cliente
    ) AS resumo
);
```

20.17 Revisão 19.5 — desafios integradores

```sql
SELECT p1.nome_produto, p1.categoria,
       p1.preco - p1.custo_aquisicao AS margem
FROM produtos AS p1
WHERE p1.preco - p1.custo_aquisicao > (
    SELECT AVG(p2.preco - p2.custo_aquisicao)
    FROM produtos AS p2
    WHERE p2.categoria = p1.categoria
);

SELECT c.nome
FROM clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM vendas AS v
    INNER JOIN produtos AS p ON v.id_produto = p.id_produto
    WHERE v.id_cliente = c.id_cliente
      AND p.preco > (SELECT AVG(preco) FROM produtos)
);

SELECT p.nome_produto
FROM produtos AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM vendas AS v
    INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
    WHERE v.id_produto = p.id_produto
      AND c.estado = 'MG'
);

SELECT p.categoria,
       SUM(v.quantidade * v.valor_unitario) AS faturamento
FROM produtos AS p
INNER JOIN vendas AS v ON p.id_produto = v.id_produto
GROUP BY p.categoria
HAVING SUM(v.quantidade * v.valor_unitario) > (
    SELECT AVG(totais.faturamento)
    FROM (
        SELECT p2.categoria,
```

```
               SUM(v2.quantidade * v2.valor_unitario) AS faturamento
        FROM produtos AS p2
        INNER JOIN vendas AS v2 ON p2.id_produto = v2.id_produto
        GROUP BY p2.categoria
    ) AS totais
);

SELECT c.nome, v1.id_venda, v1.data_venda
FROM vendas AS v1
INNER JOIN clientes AS c ON v1.id_cliente = c.id_cliente
WHERE v1.data_venda = (
    SELECT MAX(v2.data_venda)
    FROM vendas AS v2
    WHERE v2.id_cliente = v1.id_cliente
);

SELECT c.id_cliente, c.nome,
       SUM(v.quantidade * v.valor_unitario) AS total_gasto
FROM clientes AS c
INNER JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING SUM(v.quantidade * v.valor_unitario) > ALL (
    SELECT SUM(v2.quantidade * v2.valor_unitario)
    FROM clientes AS c2
    INNER JOIN vendas AS v2 ON c2.id_cliente = v2.id_cliente
    WHERE c2.estado = 'SP'
    GROUP BY c2.id_cliente
);
```

21 Checklist do estudante

Assinale cada item quando conseguir explicá-lo e produzir um exemplo próprio.

- ○ Sei identificar a consulta externa e a subconsulta.
- ○ Sei distinguir uma subconsulta independente de uma correlacionada.
- ○ Reconheço quando o resultado possui uma linha e uma coluna.
- ○ Reconheço quando o resultado possui várias linhas e uma coluna.
- ○ Reconheço quando o resultado possui uma linha e várias colunas.
- ○ Reconheço quando o resultado possui várias linhas e várias colunas.
- ○ Sei escolher entre `=`, `IN`, `ANY` e `ALL`.
- ○ Sei utilizar `EXISTS` e `NOT EXISTS`.
- ○ Entendo por que `NOT IN` exige cuidado com `NULL`.
- ○ Sei comparar tuplas com `=` e `IN`.

- ○ Sei criar uma tabela derivada com alias.
- ○ Sei usar subconsultas no `SELECT`, no `WHERE`, no `FROM` e no `HAVING`.
- ○ Entendo a diferença entre média de vendas e média de totais por cliente.
- ○ Sei identificar erros de quantidade de linhas e de colunas.
- ○ Consigo comparar soluções com subconsultas e junções.

Próxima etapa sugerida

Common Table Expressions (CTEs), views, índices, `EXPLAIN`, funções de janela e otimização de consultas analíticas.